Cálculo de distâncias 3D

PROBLEMA 13

Sequencial × paralelo

Comparação experimental em C com pthreads

Sistemas Operacionais

O problema e a saída esperada

Uma distância por ponto

$$d_i = \sqrt{((x_i - x_r)^2 + (y_i - y_r)^2 + (z_i - z_r)^2)}$$

Objetivo: calcular todas as distâncias, localizar os extremos e comparar o custo da execução sequencial com a paralela.

Entrada: N, ponto de referência e N coordenadas 3D.
Saída: vetor de distâncias, ponto mais próximo e mais distante.

Exemplo de entrada e saída

Entrada de exemplo

arquivo = entradas/pequena.txt
quantidadePontos = 20.000
referencia = (0.5, -1.25, 2.0)
pontos[0] = (-22.42160, -19.25771, 54.04782)

Saída calculada

Primeira distância: 59.6544

Ponto mais próximo
5432 (2.2416, -1.6319, 4.0617)
Distância: 2.7257

Ponto mais distante
2602 (97.3940, 94.8868, -99.3441)
Distância: 170.0040

Leitura do resultado

O programa calcula 20.000 distâncias e percorre o vetor para localizar o menor e o maior valor.

Dados reais: entradas/pequena.txt e docs/sequencial/resultados_sequencial.txt

Entradas do experimento

20.000

Pequena

120.000

Média

300.000

Grande

Os mesmos arquivos alimentam as duas versões.

Cálculo da distância euclidiana

√

Trecho literal de problema13_sequencial/sequencial.c

```
double calcularDistancia(Ponto referencia, Ponto ponto) {
    double dx = ponto.x - referencia.x;
    double dy = ponto.y - referencia.y;
    double dz = ponto.z - referencia.z;

    return sqrt(dx * dx + dy * dy + dz * dz);
}
```

A função recebe primeiro a referência e depois o ponto. O laço armazena o retorno no vetor distancias.

Implementação sequencial

Um único laço percorre todos os pontos

```
for (int i = 0; i < quantidadePontos; i++) {
    distancias[i] = calcularDistancia(referencia, pontos[i]);
}
```

Leitura e alocação precedem o cálculo.
A busca pelos extremos ocorre depois da medição.

Laço sequencial e cronometragem

Trecho literal de problema13_sequencial/sequencial.c

```
clock_gettime(CLOCK_MONOTONIC, &inicio);

for (int i = 0; i < quantidadePontos; i++) {
    distancias[i] = calcularDistancia(referencia, pontos[i]);
}

clock_gettime(CLOCK_MONOTONIC, &fim);

double tempoMs = (fim.tv_sec - inicio.tv_sec) * 1000.0
            + (fim.tv_nsec - inicio.tv_nsec) / 1000000.0;
```

O intervalo medido contém somente o cálculo das distâncias na versão sequencial.

Divisão do trabalho com pthreads

Cada thread calcula uma faixa do vetor

inicio_faixa = (n * i) / num_threads
fim_faixa = (n * (i + 1)) / num_threads

Os limites usam divisão inteira e a faixa exclui o índice fim.
As threads leem os mesmos pontos e escrevem em posições distintas.
pthread_join aguarda todas antes da busca pelos extremos.

Função executada por cada thread

Trecho literal de problema13_paralelo/dist3d_paralelo.c

```
static void *calcular_distancias(void *arg) {
    Tarefa *tarefa = (Tarefa *)arg;

    for (size_t i = tarefa->inicio; i < tarefa->fim; i++) {
        double dx = tarefa->pontos[i].x - tarefa->referencia.x;
        double dy = tarefa->pontos[i].y - tarefa->referencia.y;
        double dz = tarefa->pontos[i].z - tarefa->referencia.z;

        tarefa->distancias[i] = sqrt(dx * dx + dy * dy + dz * dz);
    }
    return NULL;
}
```

Cada Tarefa contém pontos, referencia, distancias, inicio e fim.

pthread_create, pthread_join e divisão

Trechos literais de problema13_paralelo/dist3d_paralelo.c

```
for (int i = 0; i < num_threads; i++) {
    size_t inicio_faixa = (n * (size_t)i) / (size_t)num_threads;
    size_t fim_faixa = (n * (size_t)(i + 1)) / (size_t)num_threads;
    tarefas[i].inicio = inicio_faixa;
    tarefas[i].fim = fim_faixa;
    int erro = pthread_create(&threads[i], NULL,
                              calcular_distancias, &tarefas[i]);
}

for (int i = 0; i < criadas; i++) {
    int erro = pthread_join(threads[i], NULL);
}
```

Exemplo: 12 pontos divididos entre 3 threads

[0, 4) thread 0	[4, 8) thread 1	[8, 12) thread 2

O que entra no tempo medido

Sequencial

Laço de cálculo das distâncias.

Paralelo

Criação das threads, cálculo e pthread_join.

Leitura, busca dos extremos e impressão ficam fora em ambas.

Ambiente e metodologia

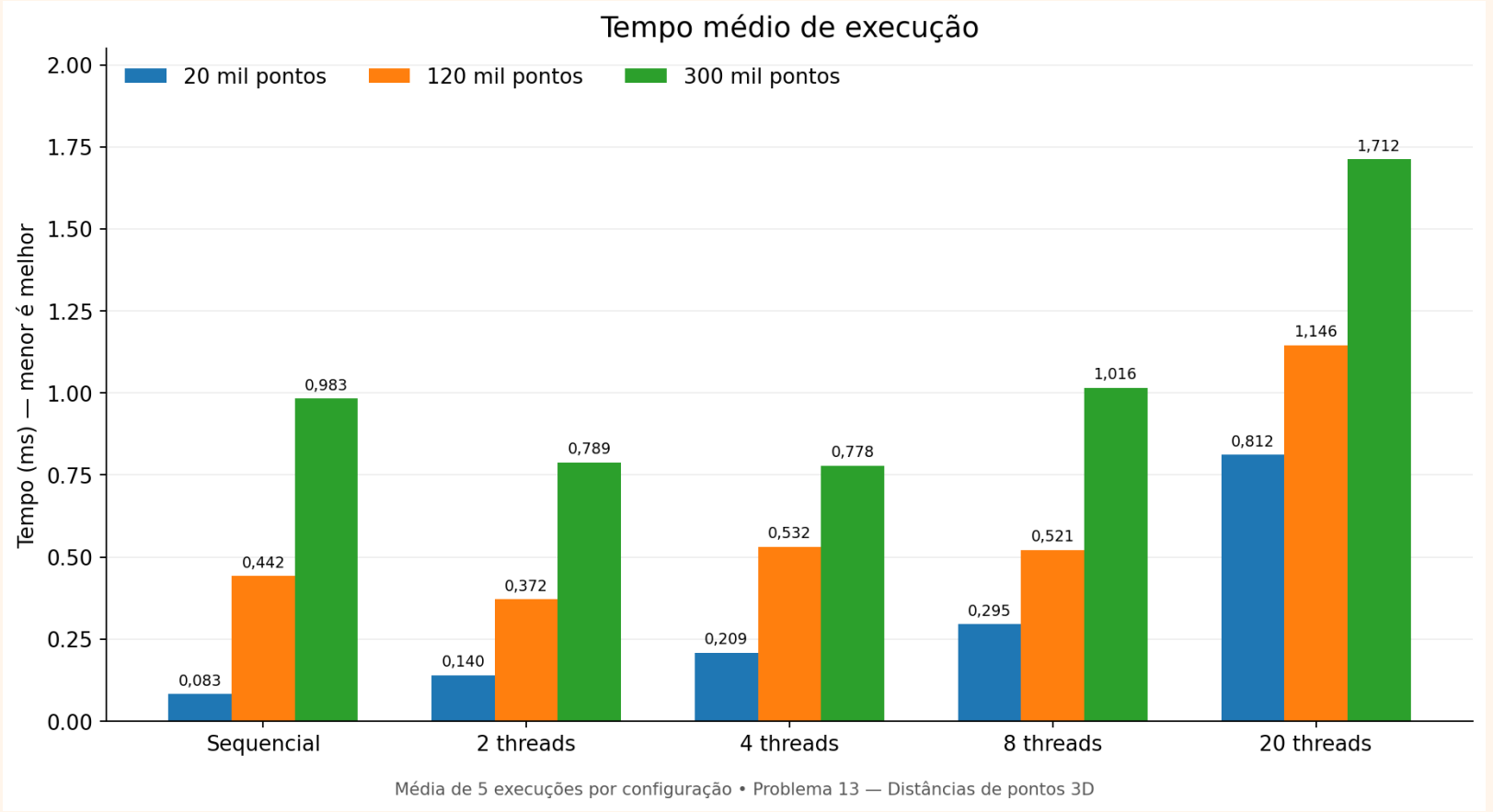
Intel Core i5-14600KF

32 GB de RAM · Ubuntu via WSL no Windows
C e GCC com otimização -O2
2, 4 e 8 threads, além de AUTO com 20 threads

5 execuções por entrada e configuração

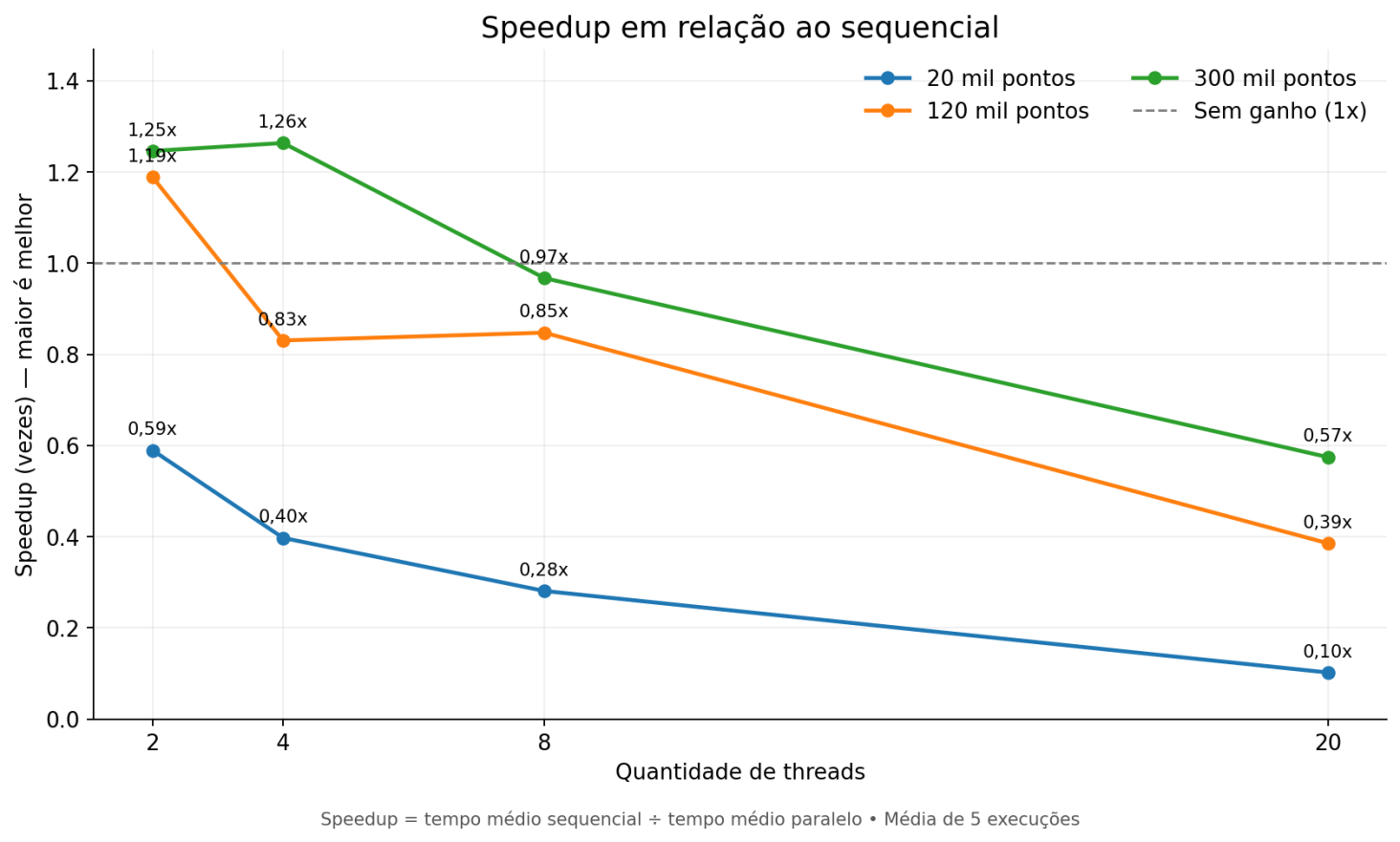
Médias em milissegundos, com todas as medições preservadas.

Tempo médio de execução



Na entrada pequena, todas as configurações paralelas foram mais lentas.

Speedup por quantidade de threads



Speedup acima de 1 indica ganho em relação ao sequencial.

Melhor resultado na entrada grande

★

1,2636×

Speedup com 4 threads

0,983127 ms no sequencial
0,778043 ms com 4 threads
31,59% de eficiência

Validação e limites dos registros

Conferência parcial dos resultados

Os extremos e as primeiras cinco distâncias coincidem nos resultados exibidos.

O resumo informa que a equivalência completa dos vetores ainda precisa de verificação programática.

Conclusões do experimento

O ganho depende da entrada e da quantidade de threads

Pequena: o sequencial apresentou o menor tempo.
Média: o melhor resultado ocorreu com 2 threads.
Grande: 4 threads tiveram o menor tempo médio.

AUTO com 20 threads foi mais lento que o sequencial nas três entradas.

OBRIGADO!

João Gnoatto
Gabriel Alvise
Gustavo Augusto
Pedro Honório

Problema 13 · Cálculo de distâncias 3D